Chapter 1
Principles of Finite Precision Computation

Numerical precision is the very soul of science.

-SIR D'ARCY WENTWORTH THOMPSON, *On Growth and Form (1942)*

There will always be a small but steady demand for error-analysts to . . .
expose bad algorithms' big errors and, more important,
supplant bad algorithms with provably good ones.

-WILLIAM M. KAHAN, *Interval Arithmetic Options in the Proposed IEEE Floating Point Arithmetic Standard* (1980)

Since none of the numbers which we take out from logarithmic and
trigonometric tables admit of absolute precision,
but are all to a certain extent approximate only,
the results of all calculations performed
by the aid of these numbers can only be approximately true . . .
It may happen, that in special cases the
effect of the errors of the tables is so augmented that
we may be obliged to reject a method,
otherwise the best, and substitute another in its place.

-CARL FRIEDRICH GAUSS', *Theoria Motus* (1809)

Backward error analysis is no panacea;
it may explain errors but not excuse them.

-HEWLETT-PACKARD, *HP-15C Advanced Functions Handbook* (1982)

[1]Cited in Goldstine [461, 1977, p. 258].

This book is concerned with the effects of finite precision arithmetic on numerical algorithms', particularly those in numerical linear algebra. Central to any understanding of high-level algorithms is an appreciation of the basic concepts of finite precision arithmetic. This opening chapter briskly imparts the necessary background material. Various examples are used for illustration, some of them familiar (such as the quadratic equation) but several less well known. Common misconceptions and myths exposed during the chapter are highlighted towards the end, in §1.19.

This chapter has few prerequisites and few assumptions are made about the nature of the finite precision arithmetic (for example, the base, number of digits, or mode of rounding, or even whether it is floating point arithmetic). The second chapter deals in detail with the specifics of floating point arithmetic.

A word of warning: some of the examples from §1.12 onward are special ones chosen to illustrate particular phenomena. You may never see in practice the extremes of behaviour shown here. Let the examples show you what can happen, but do not let them destroy your confidence in finite precision arithmetic!

1.1. Notation and Background

We describe the notation used in the book and briefly set up definitions needed for this chapter.

Generally, we use

capital letters	$A, B, C\ \Delta, \Lambda$	for matrices,
subscripted lower case letters	$\alpha_{ij}, b_{ij}, c_{ij}, \delta_{ij}, \lambda_{ij}$	for matrix elements,
lower case letters	x, y, z, c, g, h	for vectors,
lower case Greek letters	$\alpha, \beta, \gamma, \phi, \rho$	for scalars,

following the widely used convention originally introduced by Householder [587, 1964].

The vector space of all real $m \times n$ matrices is denoted by $\mathbb{R}^{m \times n}$ and the vector space of real n-vectors by $\mathbb{R}^n$. Similarly, $\mathbb{C}^{m \times n}$ denotes the vector space of complex $m \times n$ matrices.

Algorithms are expressed using a pseudocode based on the MATLAB language [232, 1988], [735, 1992]. Comments begin with the % symbol.

Submatrices are specified with the colon notation, as used in MATLAB and Fortran 90: $A(p{:}q, r{:}s)$ denotes the submatrix of A formed by the intersection of rows p to q and columns r to s. As a special case, a lone colon as the row or column specifier means to take all entries in that row or column; thus $A(:,j)$ is the jth column of A and $A(i,:)$ the ith row. The values taken by an integer

[2]For the purposes of this book an algorithm is a MATLAB program; cf. Smale [924, 1990].

variable are also described using the colon notation: "$i = 1{:}n$" means the same as "$i = 1,2, \ldots, n$".

Evaluation of an expression in floating point arithmetic is denoted $fl(\cdot)$, and we assume that the basic arithmetic operations op=+,-,*,/ satisfy

$$fl(x \operatorname{op} y) = (x \operatorname{op} y)(1+\delta), \qquad |\delta| \le u. \tag{1.1}$$

Here, u is the *unit roundoff* (or machine precision), which is typically of order 10^{-8} or 10^{-16} in single and double precision computer arithmetic, respectively, and between 10^{-10} and 10^{-12} on pocket calculators. For more on floating point arithmetic see Chapter 2.

Computed quantities (and, in this chapter only, arbitrary approximations) wear a hat. Thus $\hat{x}$ denotes the computed approximation to x.

Definitions are often (but not always) indicated by ":=" or "=:", with the colon next to the object being defined.

We make use of the floor and ceiling functions: $\lfloor x \rfloor$ is the largest integer less than or equal to x, and $\lceil x \rceil$ is the smallest integer greater than or equal to x.

The normal distribution with mean μ and variance σ^2 is denoted by $N(\mu, \sigma^2)$.

We measure the cost of algorithms in flops. A *flop* is an elementary floating point operation: +,-,/, or *. We normally state only the highest-order terms of flop counts. Thus, when we say that an algorithm for $n \times n$ matrices requires $2n^3/3$ flops, we really mean $2n^3/3+O(n^2)$ flops.

Other definitions and notation are introduced when needed. Two top ics, however, do not fit comfortably into the main text and are described in Appendix B: the singular value decomposition (SVD) and M-matrices.

All our numerical experiments were carried out either in MATLAB 4.2 [735, 1992], sometimes in conjunction with the Symbolic Math Toolbox [204, 1993], or with the Salford Software/Numerical Algorithms Group FTN90[3] Fortran 90 compiler, Version 1.2 [888, 1993]. Whenever we say a computation was "done in Fortran 90" we are referring to the use of this compiler. All the results quoted were obtained on a 486DX workstation, unless otherwise stated, but many of the experiments were repeated on a Sun SPARCstation, using the NAGWare[4] FTN90 compiler [785, 1992]. Both machines use IEEE standard floating point arithmetic and the unit roundoff is $u = 2^{-53} \approx 1.1 \times 10^{-16}$ in MATLAB and in double precision in Fortran 90. (Strictly speaking, in Fortran 90 we should not use the terminology single and double precision but should refer to the appropriate KIND parameters; see, e.g., Metcalf and Reid [749, 1990, §2.6]. However, these terms are vivid and unambiguous in IEEE arithmetic, so we use them throughout the book.)

[3]FTN90 is a joint trademark of Salford Software Ltd. and The Numerical Algorithms Group Ltd.

[4]NAGWare is a trademark of The Numerical Algorithms Group Ltd.

1.2. Relative Error and Significant Digits

Let $\hat{x}$ be an approximation to a real number x. The most useful measures of the accuracy of $\hat{x}$ are its *absolute error*

$$E_{\text{abs}}(\hat{x}) = |x - \hat{x}|,$$

and its *relative error*

$$E_{\text{rel}}(\hat{x}) = \frac{|x - \hat{x}|}{|x|}$$

(which is undefined if $x = 0$). An equivalent definition of relative error is $E_{\text{rel}}(\hat{x}) = |\rho|$, where $\hat{x} = x(1+\rho)$. Some authors omit the absolute values from these definitions. When the sign is important we will simply talk about "the error $x - \hat{x}$".

In scientific computation, where answers to problems can vary enormously in magnitude, it is usually the relative error that is of interest, because it is scale independent: scaling $x \to \alpha x$ and $\hat{x} \to \alpha\hat{x}$ leaves $E_{\text{rel}}(\hat{x})$ unchanged.

Relative error is connected with the notion of correct significant digits (or correct significant figures). The significant digits in a number are the first nonzero digit and all succeeding digits. Thus 1.7320 has five significant digits, while 0.0491 has only three. What is meant by *correct* significant digits in a number that approximates another seems intuitively clear, but a precise definition is problematic, as we explain in a moment. First, note that for a number $\hat{x}$ with p significant digits there are only $p+1$ possible answers to the question "how many correct significant digits does $\hat{x}$ have?" (assuming $\hat{x}$ is not a constant such as 2.0 that is known exactly). Therefore the number of correct significant digits is a fairly crude measure of accuracy in comparison with the relative error. For example, in the following two cases $\hat{x}$ agrees with x to three but not four significant digits by any reasonable definition, yet the relative errors differ by a factor of about 44:

$$x = 1.00000, \quad \hat{x} = 1.00499, \quad E_{\text{rel}}(\hat{x}) = 4.99 \times 10^{-3},$$
$$x = 9.00000, \quad \hat{x} = 8.99899, \quad E_{\text{rel}}(\hat{x}) = 1.12 \times 10^{-4}.$$

Here is a possible definition of correct significant digits: *an approximation $\hat{x}$ to x has p correct significant digits if $\hat{x}$ and x round to the same number to p significant digits. Rounding* is the act of replacing a given number by the nearest p significant digit number, with some rule for breaking ties when there are two nearest. This definition of correct significant digits is mathematically elegant and agrees with intuition most of the time. But consider the numbers

$$x = 0.9949, \quad \hat{x} = 0.9951.$$

According to the definition $\hat{x}$ does not have two correct significant digits ($x \to 0.99$, $\hat{x} \to 1.0$), but does have one and three correct significant digits!

or, if the data is itself the solution to another problem, it may be the result of errors in an earlier computation. The effects of errors in the data are generally easier to understand than the effects of rounding errors committed during a computation, because data errors can be analysed using perturbation theory for the problem at hand, while intermediate rounding errors require an analysis specific to the given method. This book contains perturbation theory for most of the problems considered, for example, in Chapters 7 (linear systems), 19 (the least squares problem), and 20 (underdetermined systems).

Analysing truncation errors, or discretization errors, is one of the major tasks of the numerical analyst. Many standard numerical methods (for example, the trapezium rule for quadrature, Euler's method for differential equations, and Newton's method for nonlinear equations) can be derived by taking finitely many terms of a Taylor series. The terms omitted constitute the truncation error, and for many methods the size of this error depends on a parameter (often called h, "the stepsize") whose appropriate value is a compromise between obtaining a small error and a fast computation.

Because the emphasis of this book is on finite precision computation, with virtually no mention of truncation errors, it would be easy for the reader to gain the impression that the study of numerical methods is dominated by the study of rounding errors. This is certainly not the case. Trefethen explains it well when he discusses how to define numerical analysis [1016, 1992]:

> Rounding errors and instability are important, and numerical analysts will always be the experts in these subjects and at pains to ensure that the unwary are not tripped up by them. But our central mission is to compute quantities that are typically uncomputable, from an analytic point of view, and to do it with lightning speed.

In this quotation "uncomputable" means that approximations are necessary, and thus Trefethen's point is that developing good approximations is a more fundamental task than analysing the effects of rounding errors on those approximations.

A possible way to avoid rounding and truncation errors (but not data errors) is to try to solve a problem using a symbolic manipulation package, such as Maple[5] [199, 1991] or Mathematica[6] [1109, 1991]. Indeed, we have used this approach to compute "exact answers" in some of our numerical experiments, While we acknowledge the value of symbolic manipulation as part of the toolkit of the scientific problem solver, we do not study it in this book.

[5]Maple is a registered trademark of Waterloo Maple Software.

[6]Mathematica is a registered trademark of Wolfram Research Inc.

A definition of correct significant digits that does not suffer from the latter anomaly states that *$\hat{x}$ agrees with x to p significant digits if $|x-\hat{x}|$ is less than half a unit in the pth significant digit of x.*. However, this definition implies that 0.123 and 0.127 agree to two significant digits, whereas many people would say that they agree to less than two significant digits.

In summary, while the number of correct significant digits provides a useful way in which to think about the accuracy of an approximation, the relative error is a more precise measure (and is base independent). Whenever we give an approximate answer to a problem we should aim to state an estimate or bound for the relative error.

When x and $\hat{x}$ are vectors the relative error is most often defined with a norm, as $\|x-\hat{x}\|/\|x\|$. For the commonly used norms $\|x\|_\infty := \max_i |x_i|$, $\|x\|_1 := \sum_i |x_i|$, and $\|x\|_2 := (x^T x)^{1/2}$, the inequality $\|x-\hat{x}\|/\|x| \le \frac{1}{2} \times 10^{-p}$ implies that components x_i with $|x_i| \approx \|x\|$ have about p correct significant decimal digits, but for the smaller components the inequality merely bounds the absolute error.

A relative error that puts the individual relative errors on an equal footing is the *componentwise relative error*

$$\max_i \frac{|x_i - \hat{x}_i|}{|x_i|},$$

which is widely used in error analysis and perturbation analysis (see Chapter 7, for example).

As an interesting aside we mention the "tablemaker's dilemma". Suppose you are tabulating the values of a transcendental function such as the sine function and a particular entry is evaluated as 0.124|500000000 correct to a few digits in the last place shown, where the vertical bar follows the final significant digit to be tabulated. Should the final significant digit be 4 or 5? The answer depends on whether there is a nonzero trailing digit and, in principle, we may never be able answer the question by computing only a finite number of digits.

1.3. Sources of Errors

There are three main sources of errors in numerical computation: rounding, data uncertainty, and truncation.

Rounding errors, which are an unavoidable consequence of working in finite precision arithmetic, are largely what this book is about. The remainder of this chapter gives basic insight into rounding errors and their effects.

Uncertainty in the data is always a possibility when we are solving practical problems. It may arise in several ways: from errors in measuring physical quantities, from errors in storing the data on the computer (rounding errors),